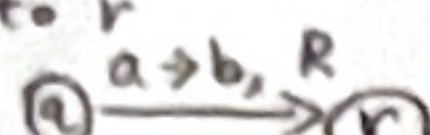


Turing Machines (TM)

1. Σ , finite input alphabet
2. Γ , finite tape alphabet (inc. Σ)
3. Q , set of states
4. q_0 , initial state
5. δ , state transition function
e.g. $\delta(q, a) = (r, b, R)$

means
if replace curr char a with b , move right by 1.
curr state change from q to r



6. q_{acc} and q_{rej} , accept state and reject states

真理

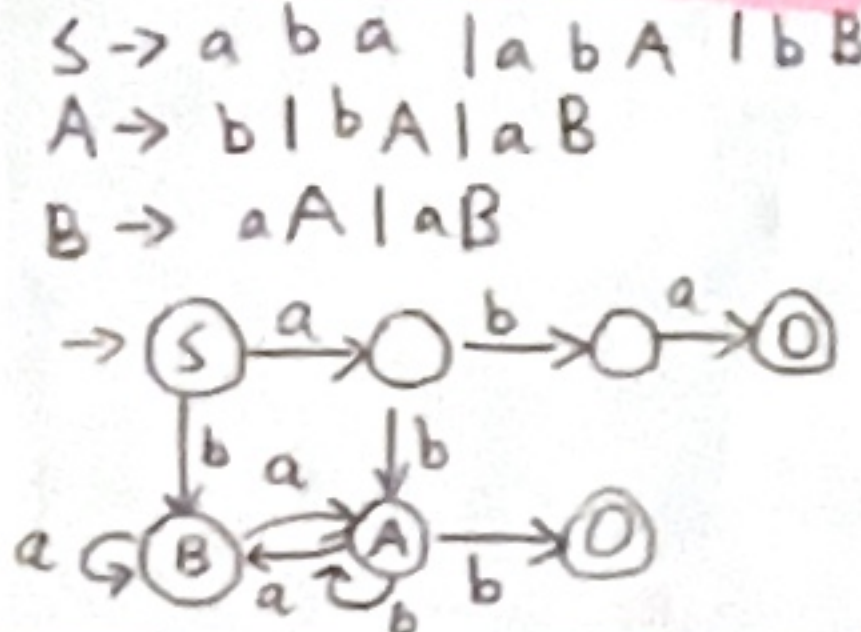
- ① machines are equivalent if and only if they recognise the same language
- ② Every NFA has an equivalent DFA
- ③ TM 进了 Accept State 就永远被 accept 了
- ④ the set of language is infinite and uncountable
- ⑤ $(P \wedge Q) \rightarrow P$ 永远 true

⑥ $P \rightarrow (P \wedge Q)$ 可变成 $(P \rightarrow P) \wedge (P \rightarrow Q)$

⑦ $A \leq (\text{reduce to}) B$ iff we can construct decider for A given decider for B

⑧ If A and A^c are both recognisable 则 A is decidable.

CFG to FA 例子



TM definitions

Halting: TM M is said to halt on input w if M eventually enters q_{acc} or q_{rej}

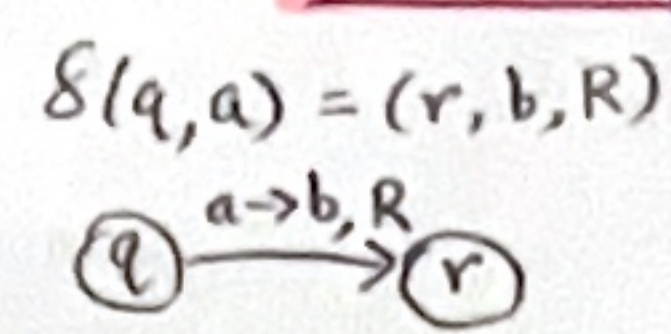
Decider: M is a decider if it halts on all inputs.

Turing-Recognisable: language A is Turing-recognisable if there exists a TM M that recognises it. $A = L(M)$

Turing-Decidable: language A is Turing-Decidable if there exists a TM M that recognises it, and M is a decider. $A = L(M)$ 且 M is decider

TM Rule
Language A is T-recognisable can be recognised by TM iff some multi-tape TM recognises A .

TM Graph



Proof: ALL DFA is decidable use decider of E_{DFA} to construct new decider for ALL DFA.

Regularity Proof: all finite languages are regular, and regular languages are closed under union, concatenation, and Kleene star. $\times$ used only these operations where sublanguage is finite or constructed using the previous mentioned operation, hence is regular.

Reduction

$$A \leq B$$

reduce to

- means A is at least as hard as B
- means if I can solve B , I can solve A too.
- means if I can't solve A , I must can't solve B too.
- if $A \leq B$ and B is decidable $\rightarrow$ then A is decidable
- if $A \leq B$ and A is undecidable, $\rightarrow$ then B is undecidable.

Fooling Set Proof

$\rightarrow$ For language $A = \{0^n 1^m\}$ suppose we have a DFA recognising A with k states. Fooling set: $\{0^m \mid 0 \leq m \leq k\}$ with $k+1$ string. For any two strings in this set, we can write one as 0^i and the other as 0^j . If we add suffix $1^i 2^i$ to both string, then $0^i 1^i 2^i \in A$ but $0^j 1^i 2^i \notin A$ since $i \neq j$. We can apply this to all pair of string.

Proof: complement of A_{TM} is undecidable.

if $\overline{A_{TM}}$ is decidable, then there must be a decider for it, H . Construct TM A such that $A =$ "on input $\langle M, w \rangle$
1. simulate H on input $\langle M, w \rangle$
2. accepts if H rejects, and vice versa."
 A halts on all input, since H is a decider. And from the behaviour we defined, A is a decider for A_{TM} , which we already know is undecidable. This completes the proof of the claim. Since A_{TM} is undecidable, we deduce that $\overline{A_{TM}}$ is undecidable as well.

Decidable Language

Closure Property

- DLs are closed under:
- Complement (接受 accept 和 reject state)
 - Union (将 $L1$ 和 $L2$ decider 的结果用 or 结合)
 - intersection (将 $L1$ 和 $L2$ decider 的结果用 and 结合)
 - concatenation (split w nondeterministically so that $w = xy$ then $MA(x)$ and $MB(y)$)
 - Kleene star (split w nondeterministically into i consecutive substrings, $w = x_1 x_2 \dots x_i$)

T-Recognisable Language

Closure Property

- RLs are closed under:
- Union
 - intersection
 - Kleene star
- RLs are not closed under:
- Complement (因为 $ACCEPT_{TM}$ 是 recognisable 但不是 decidable. 如果 A 和 A^c 都 recognisable 则 A 会是 decidable 与事实相反)

Proof: complement of $HALT_{TM}$ is not T-recognisable

From the fact that $HALT_{TM}$ is undecidable we know $\overline{HALT_{TM}}$ and $HALT_{TM}$ 都不是 T-recognisable. 所以要证明 $HALT_{TM}$ 是 T-recognisable. Consider a TM, H . On input $\langle M, w \rangle$, simulate M on w . if M accepts w or reject, then H accepts otherwise, H rejects by looping forever. So H will accept $\langle M, w \rangle$ if and only if M halts on w , and hence H recognises $HALT_{TM}$. Therefore, $\overline{HALT_{TM}}$ not recognisable. $\star A$ recogniser only need to accept all "yes" instances (and may loop forever for no instances).

Recognisable Problem Table

Machine Type	Acceptance	Emptiness	Equivalence	Halting
Regular Expression	✓	✓	✓	N/A
Context Free Grammar	✓	✓	✗	N/A
Deterministic Finite Automata	✓	✓	✓	✓
Non-deterministic Finite Automata	✓	✓	✓	✓
Pushdown Automata	✓	✗	✗	✓
Turing Machine	✓	✓	✗	✓

Decidable Problem Table

Machine Type	Acceptance	Emptiness	Equivalence	Halting
Regular Expression	✓	✓	✓	✓
Context Free Grammar	✓	✓	✗	✓
Deterministic Finite Automata	✓	✓	✓	✓
Non-deterministic Finite Automata	✓	✓	✓	✓
Pushdown Automata	✓	✓	✗	✓
Turing Machine	✗	✗	✗	✗

- Tautology : always true T
- valid : always true
- contradiction : always false F
- unsatisfiable : always false
- satisfiable : sometimes true
- non-valid : sometimes false
- contingent : sometimes true/false

$F \models G$: means G is true under model F .
also means if left side true, right side must be true.

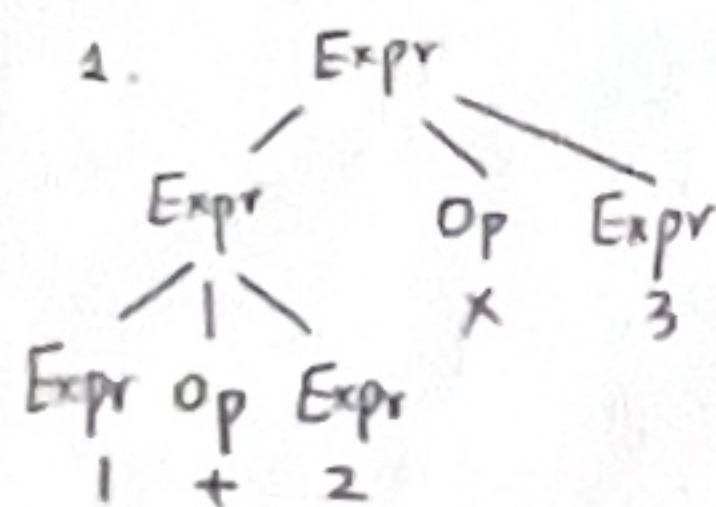
CNF: conjunctive normal form
AKA conjunction ($\wedge$)
of disjunctive clauses.
 $(P \vee Q) \wedge (\neg P)$

clausal form: $\{\{P, Q\}, \{\neg P\}\}$

Horn clause : clause with at most one positive literal.
e.g. $\neg P \vee Q$ (✓)
 $\neg P \vee Q \vee R$ (✗)

ambiguous string: a string w is ambiguous with respect to a context-free grammar if G has more than one parse tree for w .

e.g. $w = 1 + 2 \times 3$


$$2, \quad 1 + (2 \times 3)$$

Inherently ambiguous CFL : if every CFG that generates the CFL is ambiguous

Church-Turing Thesis :
every reasonable model of
unrestricted computation
is equivalent to the Turing Machine.
It also says if something is
computable $\Rightarrow$ decidable.
 $\bar{m}$ uncomputable = undecidable.

Occur (check): a variable can not appear inside its own substitution

- ① $P \rightarrow Q \equiv \neg P \vee Q$
- ② $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
 $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
- ③ $\neg P \rightarrow \neg Q \equiv Q \rightarrow P$
 $P \rightarrow \neg Q \equiv Q \rightarrow \neg P$
 $\neg P \rightarrow Q \equiv \neg Q \rightarrow P$
- ④ $P \leftrightarrow Q \equiv (P \wedge Q) \vee (\neg P \wedge \neg Q)$
 $\equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$
- ⑤ $P \rightarrow \perp \equiv \neg P$
- ⑥ $P \vee \perp \equiv P$
 $P \wedge \top \equiv P$

- (a) $P(x, a)$ and $P(b, c)$ are not unifiable
- (b) $P(f(x), y)$ and $P(a, w)$ are not unifiable as we can't be sure $f(x)$ can be a
- (c) $P(f(x), y)$ and $P(f(a), c)$ are unifiable using $\{x \rightarrow a, y \rightarrow c\}$

$$\begin{array}{ccc} \{P(x), P(y)\} & \{ \neg P(u), \neg P(v) \} \\ x \rightarrow y & & \\ \{P(x)\} & \{ \neg P(u) \} & u \rightarrow v \\ & \text{---} & x \rightarrow u \end{array}$$

例:

```
graph LR; Start(( )) --> 1((1)); 1 -- a --> 2(((2))); 2 -- b --> 1; 2 -- b --> 3((3)); 3 -- a --> 1; 2 -- "a, b" --> 2;
```

Step 1: 观察 with ϵ , 从 q_1 能去哪
 $\{1, 3\}$

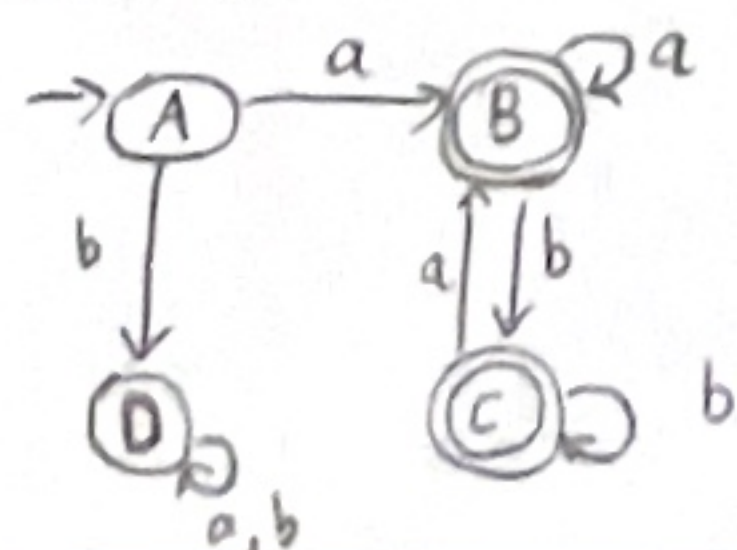
Step 2: 观察, with a , 能从 $\{1, 3\}$ reach 到 哪 $\{1, 2, 3\}$ 因为 1 能

	a	b
$A = \{1, 3\}$	B^*	
$B^* = \{1, 2, 3\}$		

... Don't forget the starting state

	a	b
$A = \{1, 3\}$	B^*	D
$B^* = \{1, 2, 3\}$	B^*	C^*
$C^* = \{2, 3\}$	B^*	C^*
$D = \emptyset$	D	D

Finally, use the table to draw the DFA:



Steps

1. Eliminate $\leftrightarrow$ and $\rightarrow$
2. Push $\neg$ in ($\forall$ 变 $\exists$, $\exists$ 变 $\forall$)
3. Rename shadowed variables

$$Y \wedge (R(x, y) \wedge \underbrace{\exists_y R(x, y)}_{\text{不同}})$$

$$\forall x (R(x, y) \wedge \exists_v R(x, v))$$
4. Eliminate $\exists$ (skolemise)
5. drop $\forall$ directly
6. Now that formula is CNF
 write it in clausal form

contains:

1. Set of States Q
2. Initial State q_0
3. State Transition
 $f(\text{curr_state}, \text{next_bit})$
 $= \text{next_state}$
4. Accept States F

Feature 1: memoryless

Feature 2: input stream
don't know when input
char is last

5 * operation: Complement, Kleene
Intersection, star
Union, are regular
Concatenation, operations

- Regular Language must be recognised by finite automata, ^{→ DFA} and must can be generated by regular expression.

- a language A is C-F
iff there is a CFG, G
that generates it.
 $A = L(G)$

- Every regular language is context-free.

- 因为 CFG 大于 RL
- a language is C.F iff can be generated

Closure Property

CFLs are closed under:

- union
- concatenation
- kleene closure
- reversal

CFLs are not closed under

- ⊗ intersection, however, if A is context-free and R is regular $A \cap R$ is context-free

• skolemise preserves
~~some~~ equisatisfiability
but not equalibility.
如果 F is satisfiable 则
skolemised F is satisfiable
unsatisfiable 同理.

• Example:

$$\begin{aligned} & \forall x ((\exists y (R(x, y) \wedge \forall z R(z, y))) \\ & \wedge (\forall u (\exists v \neg R(v, u)))) \\ & \vdash \\ & \forall x (R(x, f(x)) \wedge \forall z R(z, f(x)) \\ & \wedge (\forall u (\neg R(g(u, x), u))) \end{aligned}$$

RLs are closed under complement, intersection, union, concatenation and Kleene star. This means if

- L is regular, then L^c and L^* is also regular.

• A and B are regular, 则 $A \cap B$, $A \cup B$, $A \circ B$ 也是

- ① $E : \{E\}$
- ② $((0 \cup 1)(0 \cup 1))^*$: even length string
- ③ $((0 \cup E)(E \cup 1)) : \{E, 0, 1, 01\}$
- ④ wildcard $(0 \cup 1)^*$

CONTAINS :

1. Σ , finite set of terminals
e.g. numbers 0, 1, ..., 9
operator +, -,
parentheses (and)
2. V , finite set of variables
e.g. Op, Expr
3. R , finite set of production rule
e.g. $\text{Expr} \rightarrow 0|1|2|\dots$
 $\text{Expr} \rightarrow \text{Expr Op Expr}$
 $\text{Op} \rightarrow +$
4. S , a start variable
e.g. Expr

- an NFA with a stack

- Example:

